

Manual Junior de Desarrollo

Cómo Crear los Tests Psicométricos Nexus con HTML5 + CSS + JavaScript

OBJETIVO DEL PROYECTO

En esta práctica vais a desarrollar una plataforma sencilla de tests psicométricos utilizando:

- HTML5
- CSS3
- JavaScript Vanilla

SIN frameworks complejos.

El objetivo NO es hacer psicología real profesional, sino:

- ✓ practicar frontend
 - ✓ trabajar DOM
 - ✓ manejar formularios
 - ✓ usar arrays y objetos
 - ✓ practicar eventos
 - ✓ calcular resultados
 - ✓ crear interfaces dinámicas
 - ✓ generar gráficos y estadísticas simples
-

TESTS A DESARROLLAR

Nº	Test
1	Inteligencia Emocional
2	Valores Personales
3	Motivación Laboral
4	Estilos de Aprendizaje
5	Resolución de Conflictos

- 6 Cultura Organizacional
 - 7 Burnout
 - 8 Creatividad
 - 9 Toma de Decisiones
-

TECNOLOGÍAS

Tecnología	Uso
HTML5	estructura
CSS3	diseño
JavaScript	lógica
localStorage	guardar resultados
Flexbox/Grid	maquetación
Media Queries	responsive

ESTRUCTURA DE CARPETAS

```
project/
├── index.html
├── css/
│   └── style.css
├── js/
│   ├── app.js
│   ├── questions.js
│   ├── scoring.js
│   └── results.js
├── data/
│   └── tests.json
└── assets/
```

PASO 1 — CREAR HTML BASE

index.html

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <title>Nexus Tests</title>

  <link rel="stylesheet" href="css/style.css">
</head>

<body>

  <div class="container">

    <h1>Nexus Psychometric Tests</h1>

    <div id="question-container">

</div>

    <div class="buttons">

      <button id="prevBtn">
        Anterior
      </button>

      <button id="nextBtn">
        Siguiente
      </button>

    </div>

  </div>

  <script src="js/questions.js"></script>
  <script src="js/app.js"></script>

</body>
</html>
```

PASO 2 — CSS BÁSICO

style.css

```
body{
  font-family: Arial;
  background:#f4f4f4;
}
```

```
.container{
  width:80%;
  margin:auto;
  background:white;
  padding:20px;
  margin-top:40px;
  border-radius:10px;
}
```

```
.question{
  margin-bottom:20px;
}
```

```
button{
  padding:10px;
  cursor:pointer;
}
```

PASO 3 — CREAR PREGUNTAS EN JS

questions.js

```
const questions = [

  {
    id:1,
    text:"Puedo controlar mis emociones bajo presión",
    dimension:"regulation"
  },

  {
    id:2,
    text:"Entiendo fácilmente las emociones ajenas",
    dimension:"empathy"
  },

  {
    id:3,
```

```
text:"Necesito libertad para trabajar bien",
dimension:"autonomy"
}

];
```

EXPLICACIÓN

Cada pregunta tiene:

Propiedad	Uso
id	identificador
text	pregunta
dimension	categoría

PASO 4 — MOSTRAR PREGUNTAS

app.js

```
const container =
document.getElementById("question-container");
```

```
questions.forEach(question => {
```

```
  container.innerHTML += `
```

```
    <div class="question">
```

```
      <h3>${question.text}</h3>
```

```
      <select id="q-${question.id}">
```

```
        <option value="">Selecciona</option>
```

```
        <option value="1">1</option>
```

```
        <option value="2">2</option>
```

```
        <option value="3">3</option>
```

```
        <option value="4">4</option>
```

```
        <option value="5">5</option>
```

```
      </select>
```

```
</div>

`
});
```

¿QUÉ ESTÁ PASANDO?

Usamos:

`forEach()`

para recorrer preguntas y pintarlas en pantalla.

PASO 5 — GUARDAR RESPUESTAS

```
let answers = [];

function saveAnswers(){

  questions.forEach(question => {

    let value =
    document.getElementById(
      `q-${question.id}`
    ).value;

    answers.push({

      questionId:question.id,
      answer:Number(value),
      dimension:question.dimension

    });

  });

}
```

PASO 6 — CALCULAR SCORE

scoring.js

```
function calculateScore(){

  let total = 0;

  answers.forEach(item => {

    total += item.answer;

  });

  return total;

}
```

MOSTRAR RESULTADO

```
let result =
calculateScore();

console.log(result);
```

PASO 7 — CALCULAR POR DIMENSIONES

Ejemplo:

- empatía
- regulación
- liderazgo

```
function calculateDimensions(){

  let dimensions = {};

  answers.forEach(item => {
```

```
if(!dimensions[item.dimension]){  
    dimensions[item.dimension] = 0;  
}  
  
dimensions[item.dimension] += item.answer;  
  
});  
  
return dimensions;  
}
```

RESULTADO

```
{  
  empathy:10,  
  regulation:15,  
  autonomy:12  
}
```

PASO 8 — PREGUNTAS ESPEJO

¿QUÉ SON?

Preguntas inversas para detectar incoherencias.

EJEMPLO

Principal

"Me adapto fácilmente"

Espejo

"Me cuesta muchísimo adaptarme"

DATOS

```
{  
  id:1,  
  reverse:false  
}
```

```
{  
  id:2,  
  reverse:true  
}
```

INVERTIR RESPUESTA

```
function reverseScore(value){  
  
  return 6 - value;  
  
}
```

EJEMPLO

Original	Invertido
1	5
2	4
3	3
4	2
5	1

PASO 9 — VALIDACIÓN SIMPLE

```
function consistencyCheck(a,b){  
  
  let diff =  
    Math.abs(a-b);  
  
  return diff;  
  
}
```

RESULTADO

Diferencia	Interpretación
0	perfecta
1	buena
2	aceptable
3+	sospechosa

PASO 10 — MOSTRAR RESULTADOS

```
const resultDiv =  
document.createElement("div");  
  
resultDiv.innerHTML = `  
  
<h2>Resultado Final</h2>  
  
<p>Score: ${result}</p>  
  
`;  
  
document.body.appendChild(resultDiv);
```

PASO 11 — GUARDAR EN LOCALSTORAGE

```
localStorage.setItem(
```

```
"answers",  
JSON.stringify(answers)  
);
```

RECUPERAR

```
let saved =  
JSON.parse(  
localStorage.getItem("answers")  
);
```

PASO 12 — BARRA DE PROGRESO

HTML

```
<div class="progress">  
  
  <div id="bar"></div>  
  
</div>
```

CSS

```
.progress{  
width:100%;  
height:20px;  
background:#ddd;  
}  
  
#bar{  
width:0%;  
height:100%;  
background:green;  
}
```

JS

```
function updateBar(current,total){  
  
  let percent =  
  (current/total)*100;
```

```
document.getElementById("bar")  
.style.width = percent + "%";
```

```
}
```

PASO 13 — RESPONSIVE

Media Query

```
@media(max-width:768px){
```

```
.container{  
  width:95%;  
}
```

```
}
```

PASO 14 — DARK MODE

```
.dark{  
  background:#222;  
  color:white;  
}
```

JS

```
document.body.classList.toggle("dark");
```

PASO 15 — GENERAR PERFIL

```
function getProfile(score){
```

```
  if(score < 20){
```

```
    return "Perfil Bajo";
```

```
  }else if(score < 40){
```

```
    return "Perfil Medio";

}else{

    return "Perfil Alto";

}

}
```

PASO 16 — MINI DASHBOARD

Mostrar:

- score
- perfil
- dimensiones

```
console.log(dimensions);
```

IDEAS EXTRA

Podéis añadir:

- ✓ temporizador
 - ✓ sonidos
 - ✓ animaciones
 - ✓ múltiples tests
 - ✓ login falso
 - ✓ ranking
 - ✓ export PDF
 - ✓ gráficos
 - ✓ avatar psicológico
 - ✓ badges
-

ERRORES TÍPICOS

Error	Solución
No encuentra elemento	revisar ID
NaN	convertir Number()
undefined	revisar propiedades
no guarda	usar JSON.stringify
no carga	revisar rutas

BUENAS PRÁCTICAS

USAR FUNCIONES

 NO:

todo en un bloque gigante

 SÍ:

```
showQuestion()  
saveAnswers()  
calculateScore()
```

NOMBRES CLAROS



x
a
test1



questions

userAnswers
calculateDimensions

COMENTAR CÓDIGO

// calcular score total

OBJETIVO EDUCATIVO REAL

Con este proyecto practicáis:

Tema	Aprendido
HTML	formularios
CSS	responsive
JS	arrays
JS	objetos
JS	DOM
JS	eventos
JS	localStorage
JS	funciones
JS	condiciones
JS	bucles

NIVEL EXTRA (OPCIONAL)

Si queréis subir muchísimo el nivel:

añadir:

- fetch()
- JSON externo

- SPA simple
 - módulos JS
 - Bootstrap
 - Chart.js
 - Firebase
 - Node.js
 - PostgreSQL
-

RETO FINAL

Construir:

Nexus Psychometric Platform

con:

- ✓ menú tests
 - ✓ resultados
 - ✓ perfiles
 - ✓ dashboard
 - ✓ estadísticas
 - ✓ diseño responsive
 - ✓ validación espejo
 - ✓ scoring automático
-

CONSEJO FINAL

NO intentéis hacerlo perfecto al principio.

Primero:

1. Mostrar preguntas
2. Guardar respuestas
3. Calcular score
4. Mostrar resultados

Y DESPUÉS:

- diseño
- perfiles
- estadísticas
- mejoras

Manual Junior → Intermedio

Evolución de la Plataforma Nexus con HTML5 + CSS3 + JavaScript

OBJETIVO DEL NIVEL INTERMEDIO

En el nivel junior habéis aprendido:

- ✓ mostrar preguntas
- ✓ guardar respuestas
- ✓ calcular scores
- ✓ usar arrays y objetos
- ✓ trabajar con DOM
- ✓ usar localStorage

Ahora toca evolucionar el proyecto hacia una plataforma más profesional.

QUÉ APRENDERÉIS EN ESTE NIVEL

Tema	Nivel
Componentización JS	Intermedio
Render dinámico	Intermedio
SPA simple	Intermedio
Modularización	Intermedio
Fetch + JSON	Intermedio
Routing básico	Intermedio
Dashboard dinámico	Intermedio
Charts	Intermedio
Validaciones complejas	Intermedio
Persistencia avanzada	Intermedio

OBJETIVO FINAL

Construir una mini plataforma HRTech real:

Nexus Platform

con:

- ✓ múltiples tests
 - ✓ dashboard
 - ✓ perfiles
 - ✓ estadísticas
 - ✓ navegación dinámica
 - ✓ sistema de resultados
 - ✓ experiencia responsive
-

NUEVA ESTRUCTURA DEL PROYECTO

```
project/
├── index.html
├── css/
│   ├── style.css
│   ├── dashboard.css
│   └── responsive.css
├── js/
│   ├── app.js
│   ├── router.js
│   ├── storage.js
│   ├── scoring.js
│   ├── ui.js
│   ├── charts.js
│   └── validators.js
└── data/
    ├── emotional.json
    ├── burnout.json
    └── motivation.json
```

NIVEL 1 — CARGAR TESTS DESDE JSON

¿POR QUÉ?

En nivel junior las preguntas estaban escritas dentro del JS.

Ahora:

- ✓ los datos viven separados
 - ✓ más escalable
 - ✓ más profesional
-

emotional.json

```
[  
  {  
    "id":1,  
    "text":"Entiendo bien mis emociones",  
    "dimension":"awareness"  
  },  
  
  {  
    "id":2,  
    "text":"Puedo controlar impulsos",  
    "dimension":"regulation"  
  }  
]
```

FETCH

app.js

```
async function loadQuestions(){

  const response =
    await fetch("./data/emotional.json");

  const questions =
    await response.json();

  renderQuestions(questions);

}
```

EXPLICACIÓN

Parte	Función
fetch()	cargar archivo
await	esperar respuesta
response.json()	convertir JSON

NIVEL 2 — RENDER DINÁMICO

FUNCIÓN

```
function renderQuestions(questions){

  const container =
    document.getElementById("questions");

  container.innerHTML = "";

  questions.forEach(question => {

    container.innerHTML += `

      <div class="card">
```

```
<h3>${question.text}</h3>

<div class="answers">

  ${createScale(question.id)}

</div>

</div>

`;

});

}
```

GENERAR ESCALA AUTOMÁTICA

```
function createScale(id){

  let html = "";

  for(let i=1; i<=5; i++){

    html += `

      <button
        onclick="saveAnswer(${id},${i})"
      >
        ${i}
      </button>

    `;

  }

  return html;

}
```

¿QUÉ ESTAMOS PRACTICANDO?

- ✓ loops
 - ✓ template strings
 - ✓ funciones reutilizables
 - ✓ render dinámico
-

NIVEL 3 — SISTEMA DE RUTAS SIMPLE

OBJETIVO

Cambiar pantallas SIN recargar.

EJEMPLO

Inicio

↓

Seleccionar Test

↓

Preguntas

↓

Resultados

router.js

```
function navigate(page){

  const app =
    document.getElementById("app");

  if(page === "home"){

    renderHome();

  }

  if(page === "test"){
```

```
renderTest();  
  
}  
  
if(page === "results"){  
  
    renderResults();  
  
}  
  
}
```

NIVEL 4 — SISTEMA DE COMPONENTES

¿QUÉ ES?

Separar UI en pequeñas funciones.

EJEMPLO

Antes 

1000 líneas juntas

Después 

```
renderNavbar()  
renderQuestion()  
renderDashboard()  
renderChart()
```

EJEMPLO COMPONENTE

```
function renderNavbar(){
```

```
return `  
  
<nav>  
  
  <button onclick="navigate('home')">  
    Inicio  
  </button>  
  
</nav>  
  
`;  
}
```

NIVEL 5 — SISTEMA DE SCORING AVANZADO

SCORING POR DIMENSIÓN

```
function calculateDimensions(){  
  
  let dimensions = {};  
  
  answers.forEach(answer => {  
  
    if(!dimensions[answer.dimension]){  
  
      dimensions[answer.dimension] = 0;  
  
    }  
  
    dimensions[answer.dimension] += answer.value;  
  
  });  
  
  return dimensions;  
  
}
```

NORMALIZAR

```
function normalize(score,max){  
  
  return (score/max)*100;  
  
}
```

NIVEL 6 — VALIDACIÓN ESPEJO

JSON

```
{  
  "id":31,  
  "text":"Entiendo mis emociones",  
  "mirror":32  
}
```

VALIDACIÓN

```
function validateMirror(a,b){  
  
  const inverted =  
    6 - b;  
  
  return Math.abs(a-inverted);  
  
}
```

RESULTADO

Resultado	Significado
0	perfecta
1	muy buena

2	aceptable
3+	inconsistent e

NIVEL 7 — DASHBOARD

OBJETIVO

Mostrar:

- score
 - perfil
 - dimensiones
 - gráficos
-

HTML

```
<div id="dashboard">
```

```
</div>
```

JS

```
function renderDashboard(data){
```

```
  const dashboard =  
  document.getElementById("dashboard");
```

```
  dashboard.innerHTML = `
```

```
    <h2>${data.profile}</h2>
```

```
    <p>Score: ${data.score}</p>
```

```
  `;  
}
```

```
}
```

NIVEL 8 — CHART.JS

CDN

```
<script src="https://cdn.jsdelivr.net/npm/chart.js"></script>
```

CANVAS

```
<canvas id="myChart"></canvas>
```

GRÁFICO

```
new Chart(ctx, {  
  
  type:"radar",  
  
  data:{  
    labels:[  
      "Empathy",  
      "Regulation",  
      "Resilience"  
    ],  
  
    datasets:[{  
      data:[80,70,90]  
    }]  
  }  
  
});
```

NIVEL 9 — DARK MODE

CSS

```
.dark{  
  
background:#111;  
color:white;  
  
}
```

JS

```
function toggleDark(){  
  
document.body.classList.toggle("dark");  
  
}
```

NIVEL 10 — FILTRAR TESTS

EJEMPLO

```
const emotionalTests =  
tests.filter(test =>  
test.category === "emotional"  
);
```

¿QUÉ PRACTICÁIS?

- ✓ filter()
 - ✓ arrays
 - ✓ render dinámico
-

NIVEL 11 — LOCAL STORAGE AVANZADO

GUARDAR RESULTADOS

```
localStorage.setItem(
  "results",
  JSON.stringify(results)
);
```

HISTORIAL

```
const history =
JSON.parse(
localStorage.getItem("results")
);
```

NIVEL 12 — SISTEMA DE PERFILES

FUNCIÓN

```
function generateProfile(score){

  if(score > 80){

    return "High EQ Leader";

  }

  if(score > 60){

    return "Balanced Profile";

  }

}
```

```
return "Reactive Profile";  
}
```

NIVEL 13 — EXPORTAR RESULTADOS

OPCIÓN SIMPLE

```
window.print();
```

OPCIÓN INTERMEDIA

Usar:

- [html2pdf](#)
 - [jsPDF](#)
-

NIVEL 14 — RESPONSIVE PRO

GRID

```
.dashboard{  
  
display:grid;  
  
grid-template-columns:  
repeat(auto-fit,minmax(300px,1fr));  
  
gap:20px;  
}
```

NIVEL 15 — VALIDACIONES

NO DEJAR VACÍO

```
if(answer === ""){  
  
  alert("Responde la pregunta");  
  
}
```

VALIDAR TODO COMPLETADO

```
const completed =  
answers.length === questions.length;
```

NIVEL 16 — SPINNER DE CARGA

HTML

```
<div id="loader">  
  Cargando...  
</div>
```

JS

```
loader.style.display = "block";
```

NIVEL 17 — ANIMACIONES

CSS

```
.card{  
  
transition:0.3s;  
  
}
```

HOVER

```
.card:hover{  
  
transform:scale(1.02);  
  
}
```

NIVEL 18 — SISTEMA MULTI TEST

ESTRUCTURA

```
const tests = [  
  
  {  
    id:"eq",  
    title:"EQ Test"  
  },  
  
  {  
    id:"burnout",  
    title:"Burnout Test"  
  }  
  
];
```

SELECCIONAR TEST

```
function loadTest(testId){
```



```
fetch(`/data/${testId}.json`);  
  
}
```

NIVEL 19 — ESTADÍSTICAS

MEDIA

```
const average =  
scores.reduce((a,b)=>a+b)  
/ scores.length;
```

SCORE MÁS ALTO

```
Math.max(...scores);
```

NIVEL 20 — MODO PROYECTO REAL

ROLES

Alumno A

Frontend:

- HTML
 - CSS
 - componentes
-

Alumno B

JavaScript:

- scoring
 - validaciones
 - localStorage
-

OBJETIVO FINAL DEL NIVEL INTERMEDIO

Construir una app que tenga:

- ✓ SPA simple
 - ✓ múltiples tests
 - ✓ dashboard
 - ✓ gráficos
 - ✓ perfiles
 - ✓ validación espejo
 - ✓ responsive
 - ✓ dark mode
 - ✓ persistencia
 - ✓ navegación dinámica
-

DIFERENCIA ENTRE NIVELES

Nivel	Objetivo
Junior	hacer funcionar
Intermedio	organizar y escalar
Avanzado	arquitectura profesional

CONSEJO IMPORTANTE

NO intentéis:

- ✗ hacerlo perfecto
- ✗ meter React demasiado pronto
- ✗ usar backend complejo al inicio

Primero dominad:

- ✓ DOM
- ✓ arrays
- ✓ objetos
- ✓ eventos
- ✓ funciones
- ✓ render dinámico
- ✓ localStorage